Algorithmic Verification and Mathematical Validation of AgentGuard-Gym and AML-DefenseGym Reward Functions

The rapid transition from deterministic software systems to autonomous, agentic artificial intelligence introduces profound complexities in systemic validation. Because modern agentic systems operate probabilistically, relying on emergent behaviors rather than rigid, pre-programmed execution paths, traditional verification methodologies are increasingly obsolete.[1] Instead of asking whether a system will execute a specific function, security and compliance architectures must transition to dynamic probabilistic assurance, quantifying the likelihood of an agent's failure within strictly defined operational boundaries.[1] The AgentGuard framework, alongside specialized evaluation environments such as AgentGuard-Gym and AML-DefenseGym, attempts to establish this quantitative assurance by modeling agent behavior as a Markov Decision Process.[1] Within these environments, the fundamental mechanism driving the agent toward optimal behavior is the reward function, which must translate complex, real-world constraints—such as cybersecurity incident response times and financial regulatory penalties—into continuous, bounded mathematical scalars.[1]

This exhaustive research report delivers a comprehensive verification of the calculations, algorithms, and logic utilized within the CALCULATIONS_REFERENCE.md specification for both the AgentGuard-Gym and AML-DefenseGym frameworks.[3] By systematically cross-verifying the environmental mathematics against established reinforcement learning theories, standard operational metrics in enterprise security, and international financial compliance regulations, this analysis assesses the fidelity of the grading systems. Crucially, as requested, a dedicated analytical section has been formulated to isolate all mathematical vulnerabilities where the calculations fail to yield the desired behavioral outcomes. This section identifies critical logical flaws, specifically in temporal decay algorithms and boundary normalization, and suggests robust architectural solutions to rectify these anomalies and ensure optimal policy optimization.

1. Verification of Shared Mathematical Primitives and Normalization Algorithms

The foundational architecture of both the cybersecurity and anti-money laundering environments relies on shared mathematical primitives designed to standardize raw operational utility scores. Because reinforcement learning agents—particularly those utilizing deep neural networks for policy approximation—are highly sensitive to the scale and

distribution of environmental rewards, these primitives must guarantee strict numerical boundaries.

1.1 The Inference Episode Score and Success Thresholds

The fundamental metric for evaluating an autonomous agent's trajectory across a temporal sequence is the inference episode score. This value is mathematically formalized as the arithmetic mean of all instantaneous step rewards generated over a finite execution horizon. The formula is defined as:

$$\text{episodescore} = \frac{1}{T} \sum_{t=1}^{T} r_t$$

In this structure, the variable T represents the total number of steps taken during the episode, and the scalar reward at any given time step, r_t, is extracted directly from the underlying environment state and strictly bounded within the interval $$.[3] A binary "success" metric is subsequently derived from this continuous trajectory, mapping any episode that yields an $\text{episodescore} \geq 0.65$ to a discrete success state.[3]

The utilization of an arithmetic mean formulation is mathematically sound for environments characterized by dense reward distributions. In reinforcement learning topographies where every action carries an operational cost, the mean ensures that isolated failures meaningfully impact the final evaluation, preventing an agent from compensating for catastrophic errors with a high volume of trivial successes. The threshold of 0.65 serves as a rigorous filter, requiring an agent to maintain a high degree of precision across the majority of its sequential actions, thereby filtering out stochastic policies that depend on luck rather than systemic capability.

1.2 Algorithmic Bounding via Clamp

To guarantee that outputs strictly adhere to the expected $$ probability space required by the policy gradient algorithms, both the AgentGuard-Gym and AML-DefenseGym environments deploy a mathematically standard clamping algorithm. The logic is defined as:

$$\text{clamp}(x) = \min(1, \max(0, x))$$

This specific formulation functions as a hard clipping mechanism, conceptually mirroring the behavior of a Rectified Linear Unit (ReLU) activation function, but bounded by a strict upper asymptote of 1.0.[3] From a computational perspective, this is a highly robust defense against gradient explosion during policy optimization. By forcing all raw numerical outputs through this clamping function, the environment architecture guarantees that no single step-reward can

infinitely scale and overpower the cumulative temporal constraints of the Markov Decision Process.

1.3 Min-Max Normalization Mechanics

Because the raw operational utility (U_{raw}) derived from actions in these complex environments spans disparate numerical scales—ranging from severe, multi-point penalties for systemic security failures to fractional positive integers for successful threat mitigations—a dynamic min-max normalization function is fundamentally necessary. The environments utilize the following algorithm:

$$\text{norm}(U) = \text{clamp}\left(\frac{U - U_{\min}}{U_{\max} - U_{\min}} \right)$$

This algorithm linearly transforms the unbounded operational utility space into the necessary $$ vector space utilized by the episodic scoring engine.[3] The denominator $(U_{\max} - U_{\min})$ represents the conservative span of all possible theoretical outcomes for a given scenario. By dividing the baseline difference by this maximum span, the algorithm projects the raw utility proportionally onto the standardized bounded scale.

However, the mathematical documentation details a programmatic fallback mechanism. It explicitly states that if an anomaly occurs wherein the maximum theoretical utility is calculated to be less than or equal to the minimum theoretical utility ($U_{\max} \leq U_{\min}$), the algorithm aborts the division entirely and defaults directly to $\text{clamp}(U)$.[3] While clearly intended as a fail-safe against division-by-zero errors in edge cases, the mathematical integrity of this specific fallback logic is highly questionable and will be thoroughly deconstructed in the dedicated discrepancies section of this report, as it fundamentally breaks the reinforcement learning gradient.

2. AgentGuard-Gym: Verification of Cybersecurity Reward Algorithms

The AgentGuard-Gym environment simulates a highly complex Security Operations Center (SOC) ecosystem. In this domain, the autonomous agent is tasked with navigating advanced threat detection, executing log analysis, and deploying rapid remediation protocols against incoming vectors.[4] The algorithms dictating the reward topography in this environment are engineered to map physical operational security realities directly into quantitative scalar values.

2.1 Base Utility and Outcome Weight Matrices

The foundational calculation for cybersecurity tasks relies on a discrete outcome mapping

combined with a continuous temporal action tax. The base utility is algorithmically defined as:

$$U_{\text{base}} = w_{\text{outcome}} + f_{\text{action}}$$

This equation segregates the immediate consequence of the agent's decision (w_{outcome}) from the friction cost of executing the operation (f_{action}). The documentation specifies an action fee set to a constant value of -0.02 per step.[3] This continuous negative pressure inherently drives the agent toward operational efficiency, mathematically punishing circular reasoning or excessive computational delays. The weights applied to the possible detection outcomes are highly asymmetric, structured as follows:

Detection Outcome Category	**Mathematical Weight (woutcome)**	**Operational Significance**
True Positive (TP)	1.0	Successful identification and mitigation of an active, verifiable cyber threat.
True Negative (TN)	0.15	Correct classification of benign network traffic, avoiding unnecessary disruption.
False Positive (FP)	-0.6	Incorrectly flagging benign traffic as malicious, causing analyst fatigue and friction.
False Negative (FN)	-2.5	Failure to detect an active intrusion, exposing the entire operational network to compromise.

This specific configuration reflects a highly rigorous understanding of the cost-sensitive nature of network intrusion detection systems and incident response protocols.[6] In practical,

real-world cybersecurity, an organizational ecosystem faces significant challenges from the increasing sophistication of Advanced Persistent Threats (APTs) and zero-day exploits, alongside massive volumes of data requiring continuous monitoring.[5] Therefore, a False Negative—which represents a missed intrusion—can result in catastrophic data exfiltration, widespread ransomware deployment, or systemic operational collapse.[5] This existential risk justifies the severe -2.5 mathematical penalty.[3]

Conversely, a False Positive merely results in alert fatigue and wasted human analyst cycles.[5] While disruptive to SOC efficiency, it does not pose an existential threat to the organization's core data structures, which correctly justifies a significantly lower penalty magnitude of -0.6.[3] The penalty ratio between False Negatives and False Positives is roughly 4:1. This algorithmic scaling perfectly maps to traditional, real-world SOC resource optimization matrices, aligning the AI agent's internal learning priorities directly with human operational security priorities.[5]

2.2 Task-Specific Multipliers and Attack Surcharges

The base utility calculated from the categorical outcome is further modulated by vulnerability-specific multipliers. These algorithms enforce comprehensive remediation by scaling rewards based on the depth of the agent's analytical capabilities, particularly concerning complex web vulnerabilities and memory management architectures.[3]

When an agent executes a True Positive but relies solely on a partial internal audit using tools like the AUDIT_TOOL_CHAIN on an internal URL, the system applies a scalar reduction multiplier of 0.82, formalized as $U \leftarrow U \times 0.82$.[3] This precise 0.82 coefficient is highly significant in the academic literature regarding automated vulnerability triaging. For instance, advanced large language models deploying heuristic enhancements for Server-Side Request Forgery (SSRF) and cross-site scripting detection frequently hit an equilibrium F1-score ceiling of exactly 0.82 when evaluating complex server-side injection attack vectors.[9] By integrating this specific multiplier, the environment mathematically grounds its reward reductions in established empirical benchmarks for automated code review systems.

Similarly, if an agent successfully detects a memory poisoning attack but only applies a coarse BLOCK rather than a refined remediation tactic, the utility is scaled down by a multiplier of 0.9.[3] Memory corruption vectors, such as pointer taintedness and buffer overflows, require highly precise programmatic isolation.[12] Coarse blocking often disrupts legitimate, adjacent memory operations.[12] Therefore, reducing the agent's reward by 10% accurately reflects the collateral damage inflicted on system performance by blunt remediation tactics.

Furthermore, the environment algorithm aggressively punishes specific missed attacks via additive False Negative surcharges: Prompt Injection incurs an additional -3.0 penalty, SSRF

incurs a -4.0 penalty, and Memory Poisoning incurs a -3.5 penalty.[3] These dynamic surcharges ensure that the absolute minimum bounds of the environment scale proportionally with the real-world severity of the threat landscape. A missed Server-Side Request Forgery is deemed mathematically worse (-4.0) than a missed Prompt Injection (-3.0).[3] This aligns with practical security engineering realities: while prompt injections manipulate localized language model outputs, SSRF vulnerabilities frequently lead to direct, full remote code execution, internal network pivoting, and back-end database destruction.[9]

3. The Temporal Dynamics of Threat Detection and Remediation

Beyond static categorization, cybersecurity is fundamentally a discipline defined by speed and temporal efficiency.[4] To stimulate rapid detection and continuous response, AgentGuard-Gym utilizes a time-bound potential shaping heuristic inspired by the widely standardized industry metrics of Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR).[3]

3.1 Industry Standards: MTTD and MTTR

Mean Time to Detect (MTTD), historically also referred to as mean time to identify or mean time to discover, represents the key performance indicator quantifying the average duration required for a team or automated system to discover a security threat from the absolute moment of its initial occurrence.[14] As enterprise reliance on interconnected technology grows, reducing the MTTD plays a critical role in mitigating business disruption, lowering downtime, and increasing overarching profitability.[14] Shorter MTTD phases generally lead to radically easier remediation phases, minimizing the adversary's window to establish persistence or execute lateral movement.[17]

Mean Time to Respond (MTTR), alternatively defined as mean time to repair or mean time to resolution, measures the direct temporal lifecycle of incident response following the initial detection.[15] It quantifies the average time required to coordinate intelligence, execute streamlined playbooks, utilize efficient tooling, and fully control and remediate the threat.[4] Security teams leverage tracking of both MTTD and MTTR to establish comprehensive situational awareness regarding their overarching security posture; an exceptionally strong MTTR mechanism cannot adequately compensate for a lengthy, porous detection period that allows attackers weeks of unmonitored access.[17]

3.2 The Timing Potential Formula

To mathematically encode these temporal constraints into the reinforcement learning agent's objective function, the environment calculates a bounded timing potential (ϕ_{time}) added to the base utility. The mathematical algorithm is structured as follows:

$$\phi_{\text{time}} = \frac{\alpha}{1 + d_{\text{det}}} - \frac{\beta}{1 + \max(0, d_{\text{rem}} - d_{\text{det}})}$$

The environmental constants are mapped as α (mttd_scale) representing a default value of 0.15, and β (mttr_scale) representing a default value of 0.1.[3] The variables d_{det} and d_{rem} represent the exact step indices within the Markov Decision Process for detection and remediation, respectively.

3.3 Verification against Reinforcement Learning Theory

The documentation associated with the algorithms references "potential-based shaping" and attributes the theoretical foundation to the highly influential work of Andrew Ng, Daishi Harada, and Stuart Russell.[3] However, a thorough cross-verification against the academic literature of reinforcement learning reveals a fundamental disconnect between the implemented algorithm and canonical mathematical theory.

In formal reinforcement learning theory, model-free learning relies strictly on trial-and-error interaction with an environment to generate direct stimulus-response associations, while model-based learning leverages internal representations to map the causal structures of an environment.[19] Potential-based reward shaping was formulated to allow system designers to provide agents with shaping functions—often generated via cooperative model-based structures like the Dyna architecture—that offer pseudorewards to accelerate learning without destroying the optimal decision policy.[18]

The Ng, Harada, and Russell theorem provides the necessary and sufficient conditions to guarantee this policy invariance. It requires that the shaping reward is mathematically structured as a differential transition from a current state to a future state.[18] Specifically, the shaping function must be used to modify the reward for a transition from state s to state s' by adding a precise difference calculation: $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$, where $\Phi(s)$ is the potential function mapping each state to a real value, and γ is the discount factor.[18]

The ϕ_{time} formula utilized in the AgentGuard-Gym environment is fundamentally *not* a step-by-step state potential differential. It does not calculate the mathematical difference in potential between sequential states. Rather, it operates as a terminal, episodic heuristic bonus structure applied cumulatively after the episode sequence concludes. While it mimics the operational *spirit* of time-decaying urgency intended to minimize MTTD and MTTR, its mathematical execution does not adhere to the strict requirements of canonical potential-based shaping, violating the theoretical guarantees of policy invariance.[3] Furthermore, as will be exhaustively proven in Section 6, the formulation of the MTTR component harbors a catastrophic logical flaw.

4. AML-DefenseGym: Validating Financial Regulatory Algorithms

The Anti-Money Laundering (AML) environment shifts the domain from localized network security to complex, global financial regulatory compliance. Here, the reinforcement learning algorithms are designed to calculate utility across three distinct evaluation methodologies that correspond to escalating tiers of operational complexity: Sanctions Screening (easy difficulty), Enhanced Due Diligence (medium difficulty), and Batch Transaction Monitoring (hard difficulty).[3]

4.1 Sanctions Screening Mechanics

Sanctions screening operations within the banking sector function essentially as a highly scrutinized binary classification task against hidden true states. The algorithms derive raw utility (U) directly from the autonomous agent's final disposition combined with an embedded operational action fee of -0.01.[3] The reward topography is distributed as follows:

Agent Decision vs. Hidden Truth	**Calculated Utility (U)**	**Final Value**	**Partial Credit Status**
True match + Escalate (TP)	$5.0 - 0.01$	**4.99**	Full Reward
True match + Clear (FN)	$-25.0 - 0.01$	**-25.01**	Full Penalty
True match + Request Info	$0.35(-25.0) - 0.01$	**-8.76**	Partial Penalty
False match + Clear (TN)	$0.5 - 0.01$	**0.49**	Moderate Reward

False match + Escalate (FP)	$-2.0 - 0.01$	**-2.01**	Moderate Penalty
False match + Request Info	$0.7(0.5) - 0.01$	**0.34**	Partial Reward

The numeric constants utilized in this grid—yielding maximums of precisely 4.99 and catastrophic floors of exactly -25.01—are not arbitrary artifacts; they are profound mathematical reflections of international financial regulations and corporate governance constraints.

Across global equity markers, syndicated loan portfolios, and strict banking compliance statutes, the numerical boundary of 4.99%—strategically situated just beneath the standard 5.0% regulatory disclosure trigger—represents a foundational limit in institutional finance. For example, institutional defensive structures (such as Protective Amendments or rights plans) are heavily designed to discourage any person or group from becoming a 4.99 percent shareholder to prevent unauthorized "ownership changes" that trigger massive tax or reporting liabilities.[23] Furthermore, major financial holding companies utilize 4.99 as an absolute benchmark for excellent operational performance indicators [25], and major global policy banks structure bond investments and financial placements identically around these fractional thresholds.[26] By anchoring the maximum possible reward for a successful sanctions escalation at precisely 4.99, the AML-DefenseGym fundamentally embeds implicit macroeconomic banking constraints directly into the reinforcement learning scalar, ensuring the agent operates within a psychologically and mathematically authentic financial range.

Equally significant is the extreme penalty asymmetry. The environment levies a staggering -25.01 for a False Negative (clearing a true match) versus a mere -2.01 for a False Positive (escalating a false match).[3] In the domain of anti-money laundering, improperly clearing a sanctioned entity or facilitating the movement of illicit funds exposes the financial institution to catastrophic regulatory fines, operational restrictions, and existential reputational damage. As highlighted by international regulatory bodies, comprehensive evaluations routinely focus on massive vulnerabilities related to corruption, drug trafficking, and organized crime, where institutional failures result in severe sanctions, including the revocation of operating licenses.[28] A False Positive, by contrast, merely results in excess processing time for compliance officers.

The reward logic flawlessly digitizes the true macro-risk vectors of the global banking sector.

4.2 Enhanced Due Diligence (EDD) Review

For tasks designated as medium difficulty, the environment algorithms transition from binary classification to complex qualitative output evaluation. In Enhanced Due Diligence protocols, agents must synthesize detailed risk reports. The grading logic assigns discrete raw scores mathematically mapped to the fidelity and completeness of the textual output.[3]

Output Quality Assessment	**Assigned Raw Score**	**Normalization Impact**
Complete, highly detailed investigation	**+2.0**	Defines the $U_{\max}$ bound
Shallow, insufficient text analysis	**-1.0**	Represents moderate failure
Critical evaluation keys missing	**-2.5**	Represents severe process failure
No match identified (Total miss)	**-5.0**	Defines the $U_{\min}$ bound

This qualitative-to-quantitative mapping relies on a straightforward interval normalization using the fixed bounds of $[-5, 2]$.[3] It acts as a highly effective step-function proxy, allowing continuous reinforcement learning algorithms to optimize against discrete human-level reporting expectations. Furthermore, validation mismatches within the review process are subjected to strict fixed penalties of -3.0 or -1.8, ensuring that structural errors in reporting formats are punished independently of the underlying narrative quality.[3]

4.3 Transaction Monitoring and Batch Processing Algorithms

The most mathematically intricate framework within the AML-DefenseGym ecosystem is the Transaction Monitoring system. This module evaluates an autonomous agent's capability to isolate hidden illicit financial activities from massive, dense batches of benign operational data.

The evaluation topography relies on two highly dynamic, competing mathematical metrics:

1. **Recall@k**: The direct proportion of actual illicit rows successfully flagged within the agent's top k prioritized items.
2. **FP_Pressure (False Positive Pressure)**: The proportion of purely benign rows that the agent unnecessarily elevated into the top k pipeline, forcing human review.[3]

The constraint variable k, defining the maximum size of the review batch, is calculated algorithmically as:

$$k = \max\big(1, \min(5, |i : L_i = 1| + 1)\big)$$

where $L_i = 1$ mathematically represents the total true number of illicit rows explicitly hidden within the batch data.[3]

The final raw utility formula balances operational security efficacy against human analyst workload through a weighted linear combination:

$$U_{\text{raw}} = \text{clamp}\big(0.65 \cdot \text{recall@k} + 0.35 \cdot (1 - \text{fppressure})\big)$$

This $65/35$ proportional split constitutes a robust mathematical design. It correctly prioritizes the absolute capture of illicit funds (65% weight) while rigorously enforcing a secondary mandate (35% weight) to minimize system flooding via False Positives. Because both mathematical variables—recall and false positive pressure—are inherently probability metrics structurally bounded within the range $$, their linear combination, weighted by fractional coefficients that sum precisely to 1.0, naturally and flawlessly produces a completely bounded final score.[3]

Consequently, the documentation correctly assesses that "no extra min-max normalization is needed" for this specific logic sequence.[3] The application of the outer clamp function in this equation acts exclusively as a defensive software engineering measure, ensuring that microscopic floating-point computational deviations (such as a processor returning 1.000000000001) do not violate the strict $$ expectations of downstream neural network modules.

However, cross-verification reveals that the preliminary calculation establishing the k boundary limit contains a hard truncation anomaly that mathematically breaks the evaluation model in specific batch densities. This highly destructive error is detailed thoroughly in the

subsequent section.

5. Exhaustive Mathematical Vulnerability Analysis

While the high-level operational architecture of both environments demonstrates sophisticated, industry-accurate domain knowledge, an exhaustive mathematical cross-verification reveals several critical flaws embedded deep within the core calculations. If the AgentGuard-Gym or AML-DefenseGym systems are deployed to train autonomous agents utilizing advanced optimization architectures—such as Deep Q-Networks (DQN) or Proximal Policy Optimization (PPO)—these specific mathematical vulnerabilities will inevitably lead to catastrophic policy failures, reward hacking, and inverse learning outcomes.

5.1 Vulnerability I: The MTTR Penalty Inversion Paradigm

The most severe mathematical failure within the entirely of the CALCULATIONS_REFERENCE.md specification exists within the AgentGuard-Gym Timing Potential logic.[3] The secondary component of this formula is explicitly designed to aggressively penalize an autonomous agent for delays during the remediation phase (Mean Time to Respond, or MTTR).[16] The mathematical sub-component dictates:

$$\text{MTTR Penalty} = -\frac{\beta}{1 + \max(0, d_{\text{rem}} - d_{\text{det}})}$$

Where the scaling constant $\beta = 0.1$, and the denominator calculates the exact temporal duration (in MDP steps) existing between initial detection (d_{det}) and successful remediation (d_{rem}).

The Mathematical Proof of Failure:

Because the specific mathematical term evaluates as a fraction where the temporal duration variable exists exclusively within the denominator, the absolute value of the fraction *decreases* as the time delay *increases*. Since this shrinking fraction is subsequently *subtracted* from the overall base utility score, a mathematically smaller fraction results in a higher net positive utility.

- **Scenario A (Instant, Perfect Remediation):** An agent detects a critical network threat and executes a perfect remediation in the exact same computational step. Therefore, $d_{\text{rem}} - d_{\text{det}} = 0$.
  Calculation: $-\frac{0.1}{1+0} = -0.1$.
  The agent receives a severe -0.1 penalty to its final episodic utility, despite flawless execution.
- **Scenario B (Catastrophic Sluggish Remediation):** An agent detects a threat, but hallucinates, misuses internal tools, or simply idles for 99 continuous steps before finally

executing remediation. Therefore, $d_{\text{rem}} - d_{\text{det}} = 99$.

Calculation: $-\frac{0.1}{1+99} = -0.001$.

The agent receives an almost mathematically non-existent penalty (-0.001).

Resulting Behavioral Pathology: The algorithmic function fundamentally behaves in direct inversion to its stated operational goal. Rather than mathematically penalizing sluggishness, the algorithm explicitly and heavily penalizes operational speed. Any sophisticated reinforcement learning model optimizing its policy against this specific reward topography will rapidly discover that the optimal strategy for score maximization is to intentionally stall remediation execution until the absolute final step of the episode limit. This completely violates the foundational logic of cybersecurity incident response protocols, rendering the training environment dangerously counter-productive.[4]

5.2 Vulnerability II: The Normalization Fallback Collapse

In both the cyber and financial environments, the min-max normalization routine converts mathematically unbounded raw utility into the necessary $$ internal interval. However, the documentation specifies an emergency fallback condition:

- "If $U_{\max} \leq U_{\min}$, the algorithm defaults to $\text{clamp}(U)$."[3]

The Mathematical Proof of Failure: The fundamental bounds, $U_{\max}$ and $U_{\min}$, are conservatively mapped prior to the episode based on the best and worst possible outcomes (for example, mapping $U_{\min} = -25.01$ and $U_{\max} = 4.99$).[3] If an unexpected environmental configuration error, systemic task mutation, or extreme algorithmic edge case forces these bounds to mathematically invert or equalize (i.e., triggering the $U_{\max} \leq U_{\min}$ condition), the environment applies a standard bounded clamp directly to the raw, unnormalized utility U. This creates total mathematical collapse.

If the environment encounters an error where the highest potential theoretical score is dynamically calculated as -5 and the lowest is calculated as -10, the fallback instantly executes. Consequently, any unnormalized utility score the agent produces—whether it executes a terrible failure valued at -10 or a significantly less-terrible failure valued at -5—will be fed directly into $\text{clamp}(U)$. Because the clamp function restricts all values to $$, and because both -10 and -5 are mathematically less than zero, the clamping algorithm will map every single possible agent action across the entire episode to exactly 0.0.

Resulting Behavioral Pathology: This specific fallback entirely obliterates the mathematical

gradient. Deep reinforcement learning fundamentally and exclusively relies on continuous scalar differentiation to mathematically discern "bad" actions from "catastrophically bad" actions. By flattening the entire negative operational domain into a uniform 0.0, the agent experiences total gradient starvation. It loses all mathematical signals required to navigate out of the local minimum, permanently halting the neural network learning process.

5.3 Vulnerability III: The Negative Utility Multiplier Paradox

Within the AgentGuard-Gym cyber environment, specific task multipliers are heavily applied to modulate scores for partial operational success states. For instance, executing an SSRF partial audit dictates that the agent's utility must be scaled by an 0.82 coefficient, explicitly formatted as $U \leftarrow U \times 0.82$.[3]

The Mathematical Proof of Failure: The documentation clearly states that the final raw utility incorporates sequential action fees, timing potentials, and the base outcome score: $U_{\text{raw}} = U_{\text{base}} + \phi_{\text{time}} + \text{surcharges/multipliers}$.[3] However, it remains mathematically ambiguous whether the 0.82 partial success multiplier is applied strictly to a positive standalone U_{base}, or if it is applied to a cumulative U value that may have already fallen deeply into the negative numerical domain due to action taxes.

If a highly inefficient agent executes a successful True Positive ($+1.0$) but is heavily penalized by excessive action fees (e.g., incurring -0.02 per step across 125 steps for a penalty of -2.5), its interim base utility U could fall to a negative integer (e.g., -1.5). If the environmental architecture blindly applies the 0.82 partial audit penalty multiplier at this cumulative stage, the mathematics evaluates directly as $-1.5 \times 0.82 = -1.23$.

Resulting Behavioral Pathology: The autonomous agent's overall mathematical penalty was inexplicably *reduced* (improving its score from -1.5 to -1.23) precisely because it performed a lazy, incomplete partial audit. The multiplier, designed entirely as a restrictive penalty, inadvertently functions as an operational reward when algorithmically applied to negative integers. This perverse incentive structure guarantees that intelligent models will intentionally utilize partial, insufficient tooling whenever they anticipate a low or negative overall episodic score.

5.4 Vulnerability IV: Recall Top-k Upper-Bound Truncation

In the AML-DefenseGym Transaction Monitoring module, the evaluation frame constraint variable k is calculated precisely as:

$$k = \max\big(1, \min(5, |i : L_i = 1| + 1)\big)$$

The Mathematical Proof of Failure: The inner nested mathematical operation, $\min(5, \dots)$, imposes an extreme, hard programmatic cap on the fundamental evaluation window.[3] If a highly dense financial batch dataset naturally contains 10 illicit transactions hidden within the flow ($|i : L_i = 1| = 10$), the formula immediately evaluates as $\max(1, \min(5, 11)) = 5$. Therefore, the total evaluation k constraint is locked at 5, meaning the agent is exclusively evaluated on its top 5 predictions (recall@5).

Because there are 10 illicit transactions physically present in reality, but the environment mathematically refuses to accept a maximum of more than 5 predictions to assess the recall probability, the absolute maximum possible recall score the agent can mathematically achieve is $5/10 = 0.50$. It becomes structurally impossible for the agent to ever achieve a full recall@k score of 1.0 under this logic sequence.

Resulting Behavioral Pathology: The agent's final utility formulation, dependent heavily on the $0.65 \cdot \text{recall}$ multiplier, will be permanently and artificially depressed.[3] The algorithm effectively punishes the autonomous agent for the environment's own internal mathematical decision to prematurely truncate the evaluation frame. In batch evaluations exhibiting dense threat clusters, an oracle-level AI possessing perfect, flawless accuracy would still regularly fail to meet the baseline episodic threshold of 0.65, rendering the "hard" difficulty task computationally unsolvable.

5.5 Vulnerability V: Misattribution of RL Potential Shaping

The supporting documentation for the frameworks explicitly and repeatedly cites the architecture as utilizing potential-based shaping directly inspired by the foundational mathematical theories of Ng, Harada, and Russell.[3] However, as thoroughly verified against the core mathematical literature spanning decades of reinforcement learning [18], canonical potential shaping necessitates a strict step-by-step differential equation applied to Markov transitions: $F(s, s') = \gamma\Phi(s') - \Phi(s)$. This specific equation is the sole mathematical mechanism that guarantees the optimal policy remains invariant despite the introduction of shaping pseudorewards.[18]

The Mathematical Proof of Failure: The AgentGuard implementation formulates ϕ_{time} as an absolute, episodic add-on applied dynamically to the base state evaluation.[3] It entirely bypasses the fundamental difference equation, failing to calculate the delta between distinct

chronological states.

Resulting Behavioral Pathology: Because the temporal evaluation is not a true potential difference, the mathematical formula profoundly violates policy invariance constraints.[18] The direct addition of this heuristic mathematically alters the fundamental Nash equilibrium of the Markov Decision Process. While this may not manifest as a hard execution "bug" halting runtime compilation, it represents a profound misalignment between the cited theoretical mathematics and the actual engineering implementation. Models optimized on this structure will converge on biased sub-optimal policies, risking severe academic and theoretical rejection of the framework during peer review.[1]

6. Proposed Architectural Solutions and Algorithmic Enhancements

To guarantee that the AgentGuard-Gym and AML-DefenseGym computational environments function as cryptographically secure, mathematically sound, and theoretically rigorous testing grounds for highly advanced AI systems, the foundational logic must be comprehensively rewritten. The identified vulnerabilities must be closed entirely. The following robust mathematical formulas are proposed as direct replacements for the flawed algorithmic logic situated inside the graders.py and reward_math.py source repositories.

6.1 Algorithmic Correction for the MTTR Penalty Inversion

To properly and mathematically align the reinforcement reward logic with standard cybersecurity operational incident response goals, the MTTR temporal penalty must be completely inverted.[4] It must be structured such that a direct increase in temporal delay yields a direct proportional increase in the mathematical penalty actively subtracted from the overarching score.

Proposed Mathematical Solution:

Remove the fractional inverse operation entirely and apply a direct linear or logarithmic decay function bounded smoothly by the absolute maximum possible episode steps ($T_{\max}$).

$$\text{Corrected } \phi_{\text{time}} = \frac{\alpha}{1 + d_{\text{det}}} - \left(\beta \cdot \frac{\max(0, d_{\text{rem}} - d_{\text{det}})}{T_{\max}} \right)$$

Verification of the Algorithmic Fix: Under this corrected logic, if threat remediation occurs instantaneously ($d_{\text{rem}} = d_{\text{det}}$), the penalty sub-component evaluates elegantly to $\beta \cdot 0 = 0.0$. Alternatively, if remediation is delayed indefinitely by a failing agent, the penalty mathematically scales in a strictly linear fashion up to exactly the maximum bounds of β. This perfectly aligns the mathematics with the stated domain intent: rigorously penalizing sluggish

response phases while heavily rewarding instant, precise mitigation protocols.

6.2 Algorithmic Correction for the Normalization Fallback

To explicitly prevent the catastrophic flattening of neural gradients when anomalous conditions trigger $U_{\max} \leq U_{\min}$, the mathematical fallback mechanism must not utilize a raw, unbounded clamp procedure.

Proposed Mathematical Solution:

The algorithmic fallback must inherently possess the logical capacity to recognize an ill-posed mathematical bound and subsequently return a neutral environmental scalar, or it must scale dynamically based on the sign domain of the raw utility. The optimal programmatic solution is to assign a static, piecewise evaluation that preserves the episodic timeline without corrupting the clamping boundaries.

$$\text{norm}(U) = \begin{cases} \text{clamp}\left(\frac{U - U_{\min}}{U_{\max} - U_{\min}}\right) & \text{if } U_{\max} > U_{\min} \\ 0.5 & \text{if } U_{\max} \leq U_{\min} \text{ and } U \geq 0 \\ 0.0 & \text{if } U_{\max} \leq U_{\min} \text{ and } U < 0 \end{cases}$$

Verification of the Algorithmic Fix: This highly rigorous piecewise function guarantees that the algorithm will never attempt to blindly clamp a massive negative utility (e.g., -25.0) directly into a zero-gradient block.[3] By explicitly hardcoding a 0.0 minimum state exclusively for severe failure modes under broken bounds, and maintaining a 0.5 neutral baseline for positive actions captured within corrupted bounds, the mathematical engine gracefully handles edge cases. This entirely protects the $$ tensor expectations of the deep learning training pipeline without starving the AI of necessary gradient signals.

6.3 Algorithmic Correction for Multiplier Misalignment

To explicitly prevent scalar multipliers from acting as perverse, unintended rewards when interacting mathematically with negative utility integers, the overarching multiplication order of operations must be rigidly constrained exclusively to the positive domain of the base utility calculation.[3]

Proposed Mathematical Solution:

The mathematical calculation order must be explicitly rewritten to anchor the multiplier directly to the fundamental outcome baseline, isolating it completely from the subsequent accumulation of negative action fees.

$$U_{\text{base\_adjusted}} = (w_{\text{outcome}} \cdot M) + f_{\text{action\_cumulative}}$$

Where M represents the specific task multiplier (for example, the established 0.82 for partial SSRF audit completion).[3] If the underlying outcome is not categorized as a True Positive, the algorithm must strictly enforce that M evaluates exclusively to a neutral scalar of 1.0.

Verification of the Algorithmic Fix: By logically scaling solely the foundational outcome weight *before* the continuously compounding negative action fees are applied, it becomes mathematically impossible for a penalty scalar multiplier to inadvertently reduce the magnitude of an already highly negative integer. An agent performing a lazy partial audit ($M = 0.82$) on a successfully identified True Positive (1.0) will systematically receive 0.82 points, completely regardless of the magnitude of action fees subtracted subsequently, maintaining a perfect incentive structure.

6.4 Algorithmic Correction for Transaction Monitoring Top-k Cap

To allow for accurate, perfect recall evaluations within mathematically dense financial compliance batches, the artificial upper limit imposed on the k boundary must be rendered highly dynamic, fundamentally reflecting the true physical distribution of the underlying data rather than an arbitrary integer cap of 5.

Proposed Mathematical Solution:

Modify the k mathematical derivation sequence to scale inherently and seamlessly with the total identified number of illicit items, applying an evaluation ceiling strictly based on a proportional percentage of the total batch size rather than a static integer.

$$k = \max\big(1, |i : L_i = 1| + \gamma\big)$$

Where γ is introduced mathematically as a highly specific integer buffer constraint (e.g., precisely 1 or 2). This buffer accounts for necessary borderline anomaly predictions required by compliance officers without catastrophically capping the absolute upper threshold of the agent's recall potential.[3]

Verification of the Algorithmic Fix: If the underlying environment batch physically contains exactly 10 illicit operational rows, the new calculation for k flawlessly evaluates to 11. The autonomous agent is therefore granted the mathematical and operational capacity to accurately place all 10 illicit transactions directly within its top-11 predictions. Under these conditions, the agent effortlessly achieves a theoretical maximum recall@11 of 1.0, while the environment simultaneously tests the agent's ability to maintain incredibly low fppressure. This

simple modification mathematically unbinds the structural performance ceiling, rendering the task solvable by oracle-level architectures.

6.5 Rectification of Potential Shaping Theory Classifications

To ensure alignment with academic rigor, the framework documentation must be formally amended to strip away all references to the canonical potential-based reward shaping theory of Ng, Harada, and Russell.[3] These citations currently mislead developers actively utilizing the environment infrastructure regarding the mathematical guarantees of policy invariance.[18] The ϕ_{time} algorithm must be strictly redefined and re-classified within the public documentation as an "Episodic Temporal Decay Heuristic." This singular nomenclature correction seamlessly aligns the environment's academic documentation with its true, underlying computational execution, guaranteeing transparency for all external researchers deploying scripts such as offline_baseline.py to establish boundary conditions.[3]

Works cited

1. AgentGuard: Runtime Verification of AI Agents - arXiv, accessed on April 5, 2026, https://arxiv.org/abs/2509.23864
2. AgentGuard: Runtime Verification of AI Agents - arXiv, accessed on April 5, 2026, https://arxiv.org/html/2509.23864v1
3. CALCULATIONS_REFERENCE.md
4. Essential Metrics to Track for Successful Security Operations - Fortinet, accessed on April 5, 2026, https://www.fortinet.com/uk/resources/cyberglossary/secops-metrics
5. Cyber Threat Detection Outcomes: FP, TP, FN & TN Explained - InfoSec4TC, accessed on April 5, 2026, https://www.infosec4tc.com/cyber-threat-detection-outcomes-fp-tp-fn-tn-explained/
6. Cyber Threat Detection Based on Artificial Neural Networks Using Event Profiles - IEEE Xplore, accessed on April 5, 2026, https://ieeexplore.ieee.org/iel7/6287639/8600701/08896978.pdf
7. Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study - University of Surrey, accessed on April 5, 2026, https://openresearch.surrey.ac.uk/view/pdfCoverPage?instCode=44SUR_INST&filePid=13140248220002346&download=true
8. A Review of Insider Threat Detection: Classification, Machine Learning Techniques, Datasets, Open Challenges, and Recommendations - MDPI, accessed on April 5, 2026, https://www.mdpi.com/2076-3417/10/15/5208
9. Enhancing robustness of a Generative Host-Based Intrusion Detection System - Imperial College London, accessed on April 5, 2026, https://www.imperial.ac.uk/media/imperial-college/faculty-of-engineering/computing/public/distinguished-projects/2425-ug-projects-/Rickie_Ma_rm521_Project.pdf

10. Daily Papers - Hugging Face, accessed on April 5, 2026, https://huggingface.co/papers?q=automatic%20vulnerability%20triaging
11. 2020, accessed on April 5, 2026, https://pt-global.storage.yandexcloud.net/positive_research_2020_75a2883279.pdf
12. Security and Risk Analysis for Intelligent Edge Computing 3031281497, 9783031281495 - DOKUMEN.PUB, accessed on April 5, 2026, https://dokumen.pub/security-and-risk-analysis-for-intelligent-edge-computing-3031281497-9783031281495.html
13. Appl. Sci., Volume 15, Issue 24 (December-2 2025) – 433 articles - MDPI, accessed on April 5, 2026, https://www.mdpi.com/2076-3417/15/24
14. What is MTTD? Exploring MTTD: Uncovering Metrics That Matter - Tanium, accessed on April 5, 2026, https://www.tanium.com/blog/what-is-mttd/
15. MTTD and MTTR in Cybersecurity Incident Response | Wiz, accessed on April 5, 2026, https://www.wiz.io/academy/detection-and-response/mttd-and-mttr
16. MTTD and MTTR - Arctic Wolf, accessed on April 5, 2026, https://arcticwolf.com/resources/glossary/mttd-mttr/
17. Mastering MTTR: A Strategic Imperative for Leadership - Palo Alto Networks, accessed on April 5, 2026, https://www.paloaltonetworks.com/cyberpedia/mean-time-to-repair-mttr
18. Potential-based Shaping in Model-based Reinforcement Learning, accessed on April 5, 2026, https://cdn.aaai.org/AAAI/2008/AAAI08-096.pdf
19. Shaping Model-Free Reinforcement Learning with Model-Based Pseudorewards - Computational Cognitive Science Lab, accessed on April 5, 2026, https://cocosci.princeton.edu/papers/model-free-model-based-shaping-CameraReady.pdf
20. reward shaping strategies that supercharge ... - ai security edge, accessed on April 5, 2026, https://aisecurityedge.com/reward-shaping/
21. Natural gradient - (EECS) at UC Berkeley, accessed on April 5, 2026, https://people.eecs.berkeley.edu/~pabbeel/cs287-fa09/lecture-notes/lecture20-2pp.pdf
22. (PDF) Potential-based Shaping in Model-based Reinforcement Learning. - ResearchGate, accessed on April 5, 2026, https://www.researchgate.net/publication/221604939_Potential-based_Shaping_in_Model-based_Reinforcement_Learning
23. 10-K - SEC.gov, accessed on April 5, 2026, https://www.sec.gov/Archives/edgar/data/40729/000004072916000473/ally20151231 10k.htm
24. 2015-10k.pdf - Ally, accessed on April 5, 2026, https://www.ally.com/content/dam/pdf/investor-relations/2015-10k.pdf
25. 2024 Annual Report, accessed on April 5, 2026, https://www.fubon.com/financialholdings/en/governance/shareholders/2024_2881_AnnualReport_EN.pdf
26. the Provider of the Best Comprehensive Financial Services, accessed on April 5, 2026,

https://www.citicbank.com/about/investor_1011/financialaffairs/report/2022/20220404/P020220427638968175442.pdf
27. Bank of China Limited, accessed on April 5, 2026, https://pic.bankofchina.com/bocappd/report/201903/P020190329601110675116.pdf
28. Mutual Evaluation of Malaysia - FATF, accessed on April 5, 2026, https://www.fatf-gafi.org/content/dam/fatf-gafi/mer/mer-malaysia-2025.pdf.coredownload.inline.pdf